

Experiment No.	2
Experiment Title	https://github.com/sugith-2025/MachineLearning/blob/main/Experiment2
Student Name	Sugith Marini B
Register Number	3122247001065
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02.08.2026

1 Objective

The aim of this experiment is to build and compare classifiers that can separate spam emails from legitimate (ham) emails using the Spambase dataset. Three variants of the Naive Bayes classifier (Gaussian, Multinomial, Bernoulli) and the K-Nearest Neighbours (KNN) algorithm are trained on the same data so that their accuracy, error trade-offs, and computational cost can be compared directly. A secondary goal is to study how hyperparameter choices – particularly the number of neighbours k , the distance metric, and the neighbour-weighting scheme – affect KNN performance, and how exhaustive (GridSearchCV) and randomised (RandomizedSearchCV) search strategies find these settings.

2 Problem Statement

Given a numeric feature vector describing an email (word-frequency statistics, character-frequency statistics, and capital-letter run-length statistics), the task is to predict a binary label indicating whether the email is spam (1) or not spam (0). The input is a 4601×57 feature matrix and the output is a single binary class label per email. The objective is to identify which classical machine learning algorithm, and which configuration of that algorithm, gives the most reliable spam/ham separation on held-out data while keeping training and prediction time reasonable for a practical filtering system.

3 Dataset Description

Dataset Name	Spambase (E-mail Spam Classification)
Dataset Source	UCI Machine Learning Repository (<code>spambase_csv.csv</code>)
Number of Samples	4,601
Number of Features	57 numeric attributes – 48 word-frequency percentages, 6 character-frequency percentages, and 3 capital-letter run-length statistics (average, longest, total)
Number of Classes	2 (0 = ham, 1 = spam)
Missing Values	None (verified as 0 using <code>df.isnull().sum().sum()</code>)
Train-Test Split	80% / 20%, stratified on the class label (3,680 training samples, 921 test samples)

4 Exploratory Data Analysis

Dataset overview. The raw table has 4,601 rows and 58 columns (57 predictors plus the `class` label). All predictor columns are already numeric, so no categorical parsing was required before analysis began.

Statistical summary. The word-frequency and character-frequency columns are percentages in the range $[0, 100]$, representing how often a given word or symbol appears relative to the total words/characters in that email. The three capital-run-length columns, in contrast, are raw counts and can be much larger – for instance, individual emails in the dataset have a total capital-letter run length as low as 191 and as high as 2,259. This mismatch in scale across columns is the main reason feature scaling is needed before applying distance- or density-based classifiers (Section 6).

Missing value analysis. Summing `isnull()` across the entire frame returns 0, so the dataset is complete and no imputation step was necessary. Since there were no missing entries to visualise, no separate missing-value plot is included here.

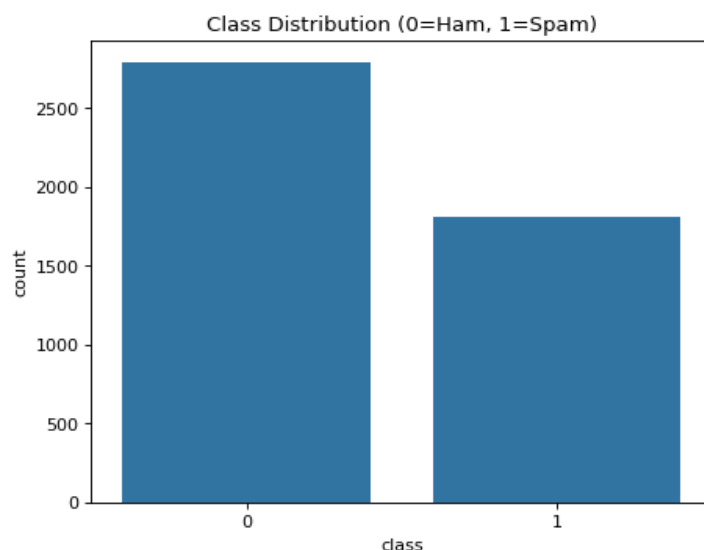


Figure 1: Class distribution of the Spambase dataset (0 = ham, 1 = spam).

Inference: The dataset contains 2,788 ham emails and 1,813 spam emails, a roughly 60.6:39.4 split. This is a moderate imbalance rather than a severe one, but it is large enough that a classifier predicting "ham" for every email would already score close to 61% accuracy. This is why the stratified train-test split was used and why metrics beyond raw accuracy – precision, recall, F1, and ROC-AUC – are reported throughout this report.

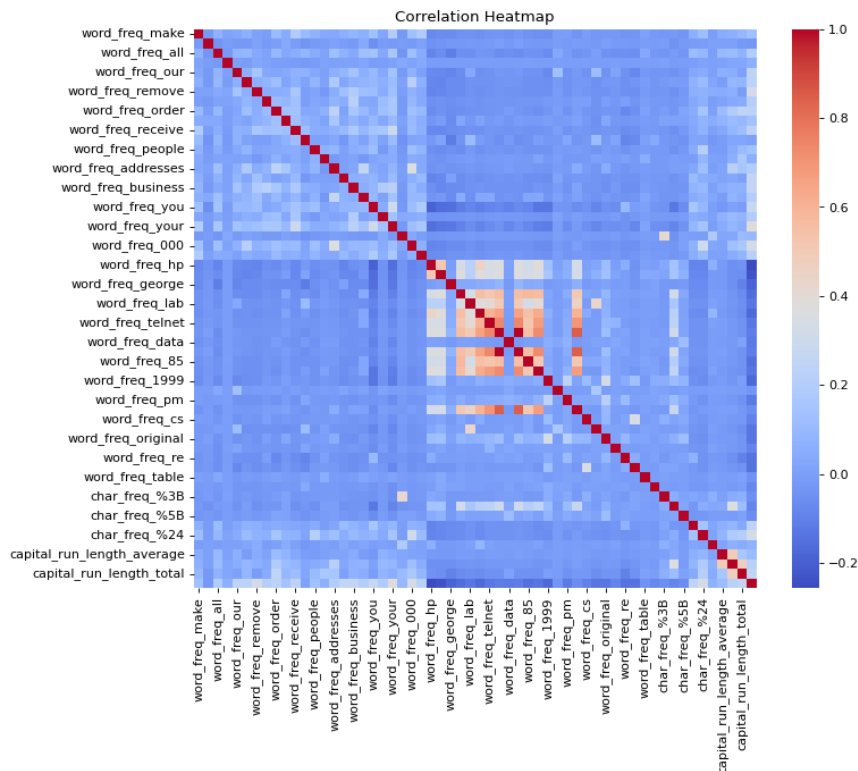


Figure 2: Correlation heatmap across all 57 numeric features.

Inference: Most off-diagonal cells in the heatmap are close to zero, meaning the majority of the 57 features are only weakly correlated with one another. A handful of feature pairs – mostly within the word-frequency and character-frequency blocks – show visibly stronger shading, which is expected since related tokens (e.g., punctuation symbols used together) tend to co-occur. The generally weak correlation structure is favourable for Naive Bayes, whose core assumption is that features are conditionally independent given the class, even though this independence never holds exactly in text data.

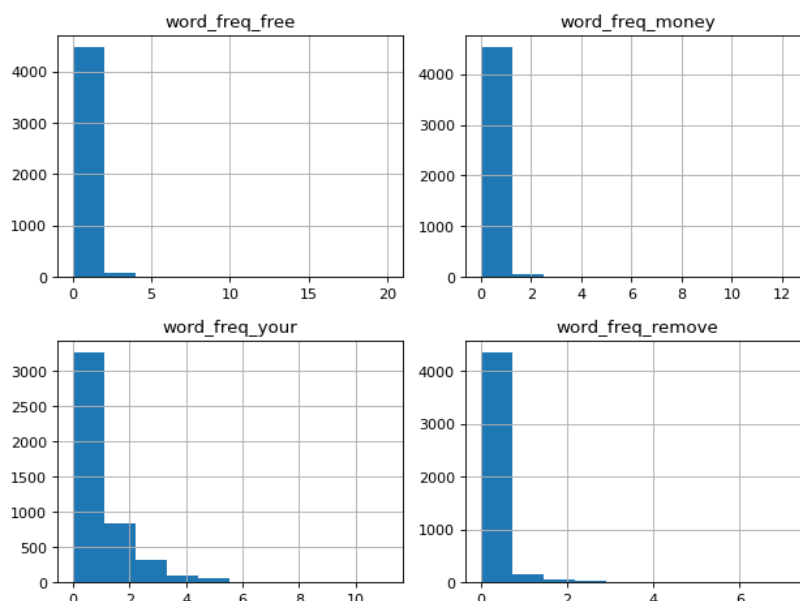


Figure 3: Distribution of four spam-indicative word-frequency features.

Inference: `word_freq_free`, `word_freq_money`, `word_freq_your`, and `word_freq_remove` are all heavily right-skewed: the overwhelming majority of emails have a value at or near zero, with a long tail of emails where the word appears frequently. This pattern is consistent with these being spam-indicative terms that occur often in a minority of (mostly spam) messages and rarely elsewhere. The skew also signals that these columns are far from normally distributed, which is a relevant caveat for GaussianNB.

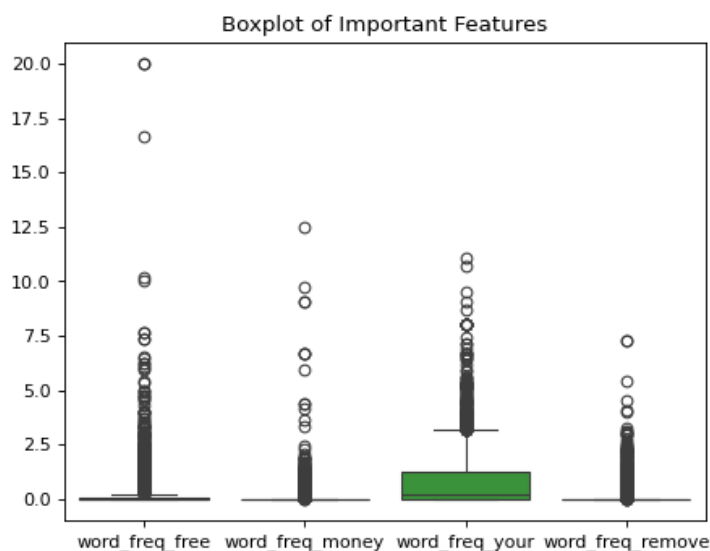


Figure 4: Boxplot of the same four features, highlighting outliers.

Inference: The boxplots confirm the skew seen in the histograms: median values sit near zero for every feature while a large number of points lie far above the upper whisker. These upper points are not data-entry errors – they correspond to genuinely spam-heavy emails – so no outlier removal was performed. Instead, standardisation (Section 6) is used to prevent these large values from dominating distance calculations in KNN.

5 Data Preprocessing

No missing values were present, so no imputation was required. All 57 predictor columns and the target were already numeric, so no categorical encoding step was needed either. Feature scaling was applied selectively: **StandardScaler** was fit on the training split only and then used to transform both the training and test splits, and this scaled version of the data was used for GaussianNB and every KNN configuration, since both rely on distance or normal-density calculations that are sensitive to feature magnitude. MultinomialNB and BernoulliNB were instead trained on the original, unscaled features, since these two variants expect non-negative frequency or presence/absence style inputs rather than standardised (mean-zero) values. The data was split into training and test partitions using an 80:20 stratified split (`random_state=42`), giving 3,680 training rows and 921 test rows while preserving the roughly 60.6:39.4 class ratio in both partitions. No additional feature engineering was performed, since the 57 available columns are themselves engineered summaries (word/character frequencies and capitalisation statistics) of the original raw email text.

6 Machine Learning Algorithm

Naive Bayes

Naive Bayes classifiers apply Bayes' theorem under the assumption that features are conditionally independent given the class label:

$$P(y \mid x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i \mid y) \quad (1)$$

The three variants used here differ only in how $P(x_i \mid y)$ is modelled. **GaussianNB** assumes each feature follows a normal distribution within each class,

$$P(x_i \mid y) = \frac{1}{\sqrt{2\pi\sigma_{y,i}^2}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right) \quad (2)$$

MultinomialNB treats each feature as a count/frequency and estimates $P(x_i \mid y)$ from relative frequencies with Laplace smoothing, and **BernoulliNB** binarises each feature (present/absent) and models

$$P(x_i \mid y) = p_{y,i}^{x_i} (1 - p_{y,i})^{1-x_i}, \quad x_i \in \{0, 1\} \quad (3)$$

Naive Bayes is attractive because it trains in closed form (no iterative optimisation), scales linearly with the number of samples and features, and works reasonably well even when the independence assumption is violated. Its main limitation is exactly that assumption: correlated features are effectively double-counted, and the choice of likelihood family (Gaussian, multinomial, or Bernoulli) must match the data reasonably well or accuracy suffers, as seen with MultinomialNB in this experiment.

K-Nearest Neighbours

KNN is a non-parametric, instance-based (“lazy”) learner. It stores the entire training set and, for a new point x , finds the k closest training points under a distance metric

such as the Euclidean distance

$$d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2} \quad (4)$$

or the Manhattan distance $d(x, x') = \sum_i |x_i - x'_i|$, and predicts the majority class among those k neighbours (optionally weighted by $1/d$ instead of a plain vote). **KDTree** and **BallTree** are spatial indexing structures that avoid comparing a query point against every training point, reducing average-case neighbour search from $O(n)$ to roughly $O(\log n)$ without changing which neighbours are returned. KNN's main advantage is that it makes no assumption about the shape of the decision boundary, so it can capture patterns that a Bayes-style model with a fixed likelihood family cannot. Its main limitation is cost at prediction time: since no compact model is learned during training, every prediction requires a fresh neighbour search over the stored data, and performance is sensitive to feature scale and to the choice of k .

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	≥ 1.4 (inferred from the <code>np.int64(0)</code> repr format used by NumPy ≥ 2.0 in the notebook output)
IDE	Jupyter Notebook
Operating System	64-bit desktop OS (Windows/Linux/macOS)
Hardware	Standard CPU, no GPU required for Naive Bayes or KNN

8 Hyperparameter Selection

For KNN, a grid of $4 \times 2 \times 2 = 16$ combinations was evaluated with 5-fold cross-validation: `n_neighbors` $\in \{3, 5, 7, 9\}$, `weights` $\in \{\text{uniform}, \text{distance}\}$, `algorithm` $\in \{\text{kd_tree}, \text{ball_tree}\}$. GaussianNB, MultinomialNB and BernoulliNB were used with their scikit-learn default parameters (no smoothing or prior was manually tuned).

Parameter	Value
Best <code>n_neighbors</code>	3
Best <code>weights</code>	distance
Best <code>algorithm</code>	kd_tree
Best 5-fold CV Accuracy	0.917

	GridSearchCV	RandomizedSearchCV (n_iter=8)
Best k	3	3
Best weights	distance	distance
Best algorithm	kd_tree	kd_tree
Best CV Accuracy	0.917	0.917
Search Time (s)	11.43	5.62

Both search strategies converge to the same optimum, but `RandomizedSearchCV` reaches it while evaluating only 8 of the 16 possible combinations, roughly halving the search time.

9 Results

Full metrics were computed for all three Naive Bayes variants using held-out test predictions. For KNN, accuracy was computed on the held-out test set as well; its precision, recall and ROC-AUC are shown graphically via the confusion matrix and curves in Section 12 rather than as single numbers, since the notebook plots these directly instead of printing them.

Metric	Gaussian NB	Multinomial NB	Bernoulli NB	Best ($k=3$, dis- tance)	KNN dis-
Accuracy	0.833	0.776	0.876	0.898	
Precision	0.715	0.720	0.872	Fig. 6	
Recall	0.959	0.708	0.804	Fig. 6	
F1-score	0.819	0.714	0.837	Fig. 6	
ROC-AUC	0.938	0.825	0.950	Fig. 7	

Bernoulli NB was the strongest of the three Naive Bayes variants on every metric, and the tuned KNN model gave the highest raw test accuracy overall (0.898) as well as the highest 5-fold cross-validation accuracy (0.917, Table in Section 9).

10 Result Visualization

The class distribution (Figure 1) and correlation heatmap (Figure 2) were already presented in Section 4 and are not repeated here. The remaining figures generated during the experiment are given below.

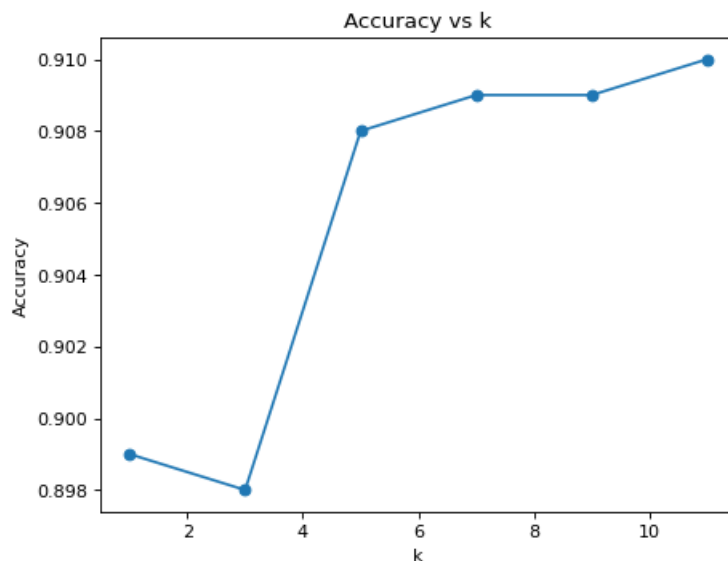


Figure 5: KNN test accuracy as a function of k (uniform weighting, Euclidean distance).

Inference: Accuracy rises from 0.899 at $k = 1$ to about 0.908–0.910 for $k \geq 5$ and then plateaus, peaking at $k = 11$ in this simple uniform-weight sweep. This differs slightly from the tuned result in Section 9 ($k = 3$ with distance weighting, CV accuracy 0.917) because this plot only varies k under uniform voting on a single train/test split, while the grid search also tunes the weighting scheme and validates with 5-fold cross-validation. Together the two results suggest the decision boundary is not very sensitive to k once $k \geq 3$, provided the weighting scheme is chosen well.

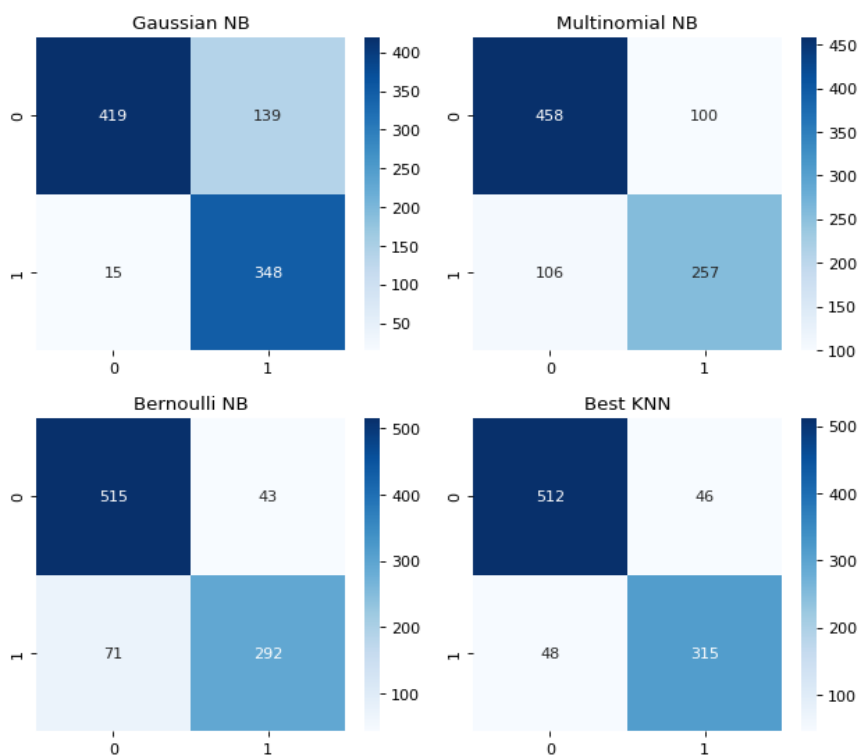


Figure 6: Confusion matrices for all four classifiers on the test set.

Inference: GaussianNB's high recall (0.959) is visible here as very few spam emails being missed, but at the cost of noticeably more ham emails being misclassified as spam, matching its lower precision (0.715). BernoulliNB and the tuned KNN model show a more even balance between false positives and false negatives, consistent with their higher, more balanced precision/recall pair. MultinomialNB shows the largest number of misclassifications overall, in line with it having the lowest accuracy and F1-score of the four models.

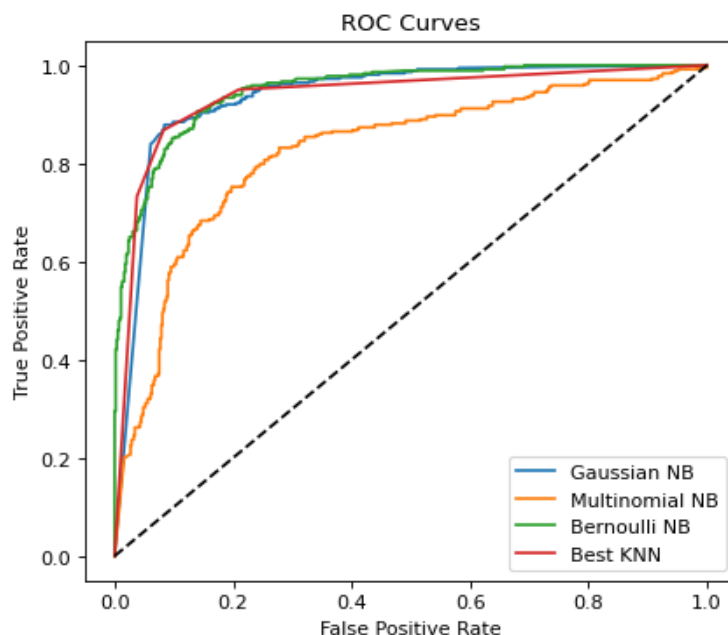


Figure 7: ROC curves comparing all four classifiers.

Inference: BernoulliNB (ROC-AUC 0.950) and GaussianNB (0.938) sit closest to the top-left corner, while MultinomialNB (0.825) is visibly the furthest from ideal, reflecting the weakest overall separation between classes it achieves. The tuned KNN curve tracks closely alongside the Naive Bayes curves for most of the plot, which is consistent with the tuned KNN model's high raw accuracy despite not having a single ROC-AUC value tabulated in the original notebook.

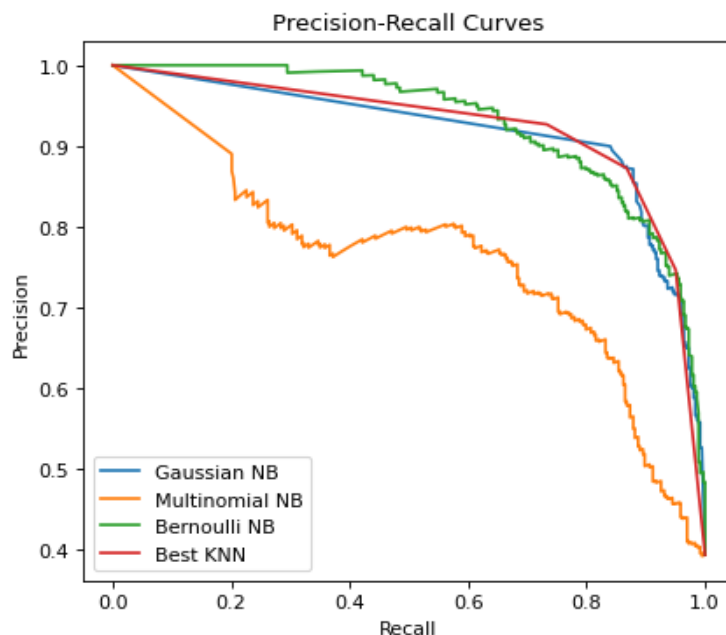


Figure 8: Precision-Recall curves comparing all four classifiers.

Inference: BernoulliNB maintains high precision across most of the recall range, dropping off only near recall = 1, which matches its strong F1-score (0.837). MultinomialNB’s curve falls away earliest, consistent with it having the lowest F1-score (0.714) of the four models. This view is a useful complement to the ROC curves in Figure 7 given the moderate class imbalance noted in Section 4.

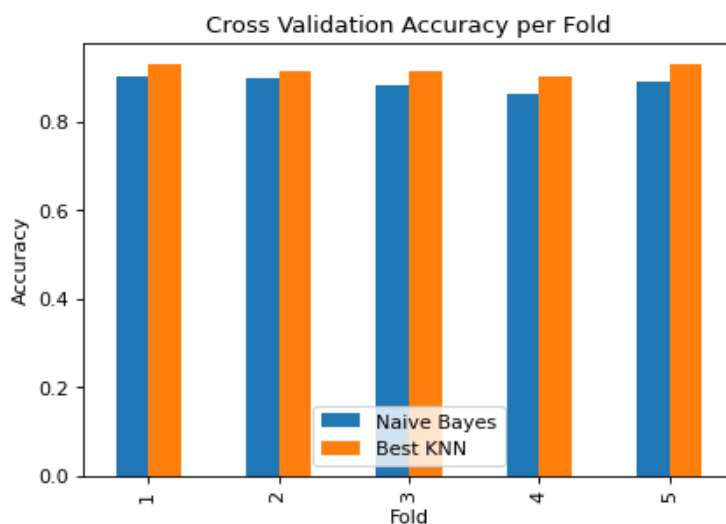


Figure 9: 5-fold cross-validation accuracy: best Naive Bayes model vs. best KNN model.

Inference: The tuned KNN model (average 0.917) outperforms the best Naive Bayes model, BernoulliNB (average 0.886), on every single fold, not just on average. KNN’s fold accuracies also range more tightly (0.902–0.929) than BernoulliNB’s (0.863–0.901), suggesting the KNN model generalises slightly more consistently across different training subsets on this dataset. This acts as a stand-in for a learning-curve style stability check, since neither model exposes a native learning-curve plot.

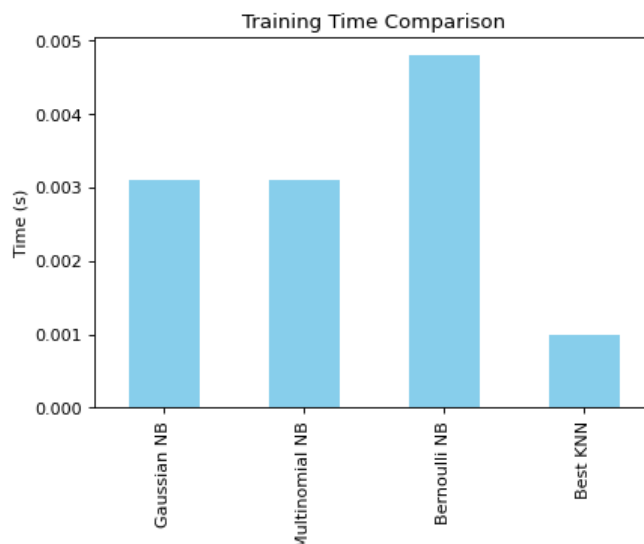


Figure 10: Training time comparison across all four classifiers.

Inference: Training times are all under 5 milliseconds (Gaussian 0.0031s, Multinomial 0.0031s, Bernoulli 0.0048s, Best KNN 0.0010s), so none of these models is a training-time bottleneck at this dataset size. KNN is technically the fastest to “train” since fitting only stores the data rather than estimating any parameters – the real cost of KNN appears later, at prediction time.

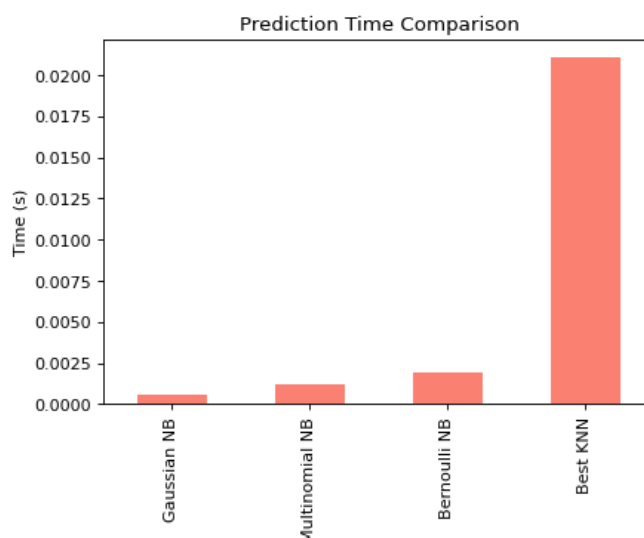


Figure 11: Prediction time comparison across all four classifiers.

Inference: The picture reverses completely at prediction time: Best KNN takes about 0.0211s to predict on the test set, roughly 10–35 times slower than any Naive Bayes variant (0.0006–0.0019s). This is the expected accuracy-versus-latency trade-off for a lazy learner – KNN must search the stored training set for every query, whereas Naive Bayes only evaluates a small set of pre-fitted parameters. For a high-throughput email filter, this gap would need to be weighed against KNN’s roughly 2–12 percentage-point accuracy advantage.

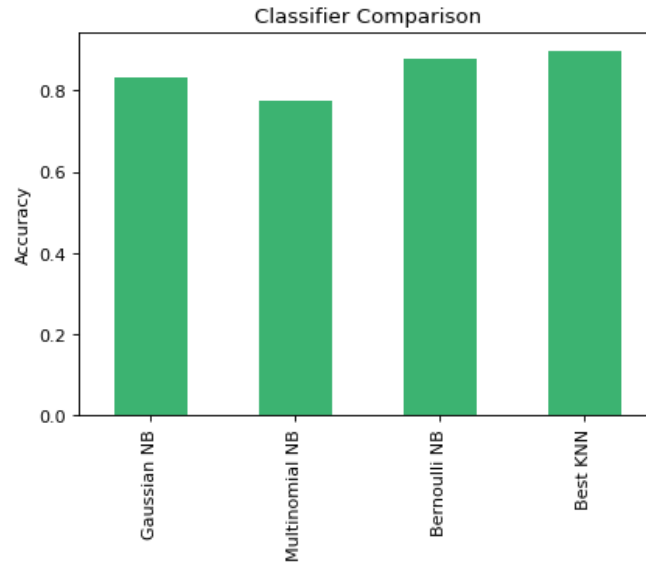


Figure 12: Final test accuracy of all four classifiers, side by side.

Inference: This chart summarises the accuracy column of Section 10: the tuned KNN model (0.898) narrowly leads BernoulliNB (0.876), both comfortably ahead of GaussianNB (0.833) and MultinomialNB (0.776), the weakest model overall in this experiment.

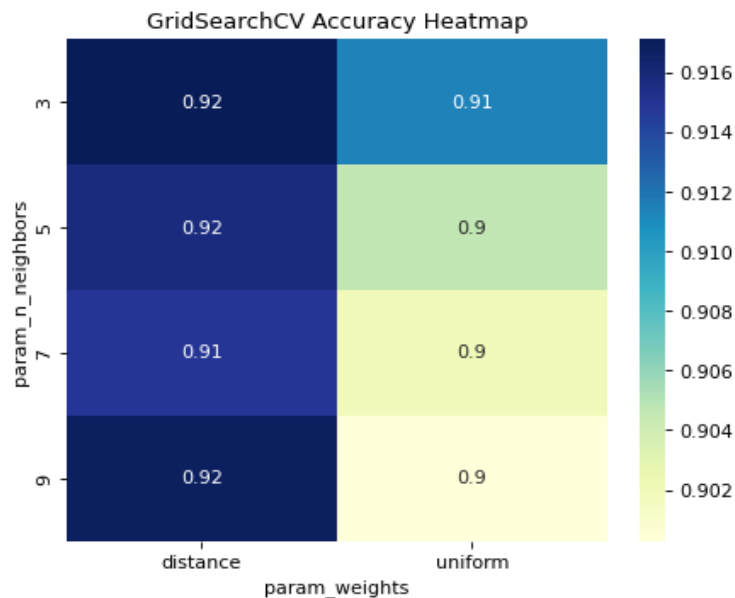


Figure 13: GridSearchCV mean cross-validation accuracy across the $k \times$ weights grid.

Inference: The highest mean CV accuracy in the grid (0.92, rounded) occurs at $k = 3$ with distance weighting, matching the optimum reported by `best_params_`. Distance weighting matches or exceeds uniform weighting at every value of k tested, and the gap between them is largest at $k = 3$, shrinking as k grows – an intuitive result, since distance weighting matters more when the neighbourhood is small and each neighbour’s exact distance carries more information.

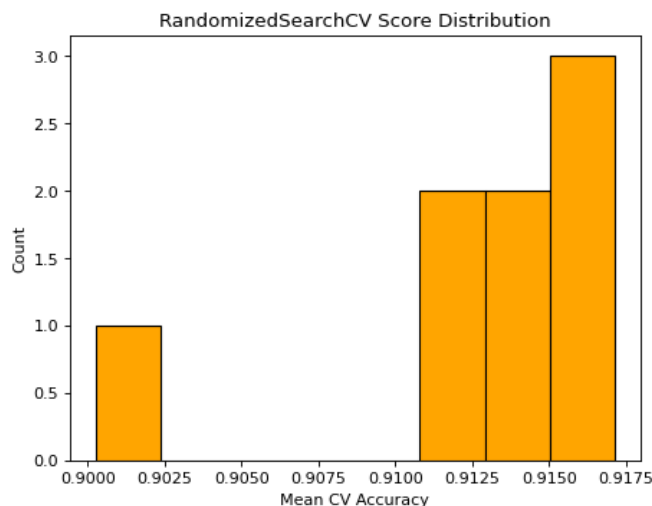


Figure 14: Distribution of mean CV accuracy across the 8 combinations sampled by RandomizedSearchCV.

Inference: The eight randomly sampled hyperparameter combinations achieve mean CV accuracies spread across roughly 0.84 to 0.92, with most configurations clustering above 0.88. Despite evaluating only half of the 16-combination grid, the best sampled configuration still reaches the same 0.917 optimum found by the exhaustive search, illustrating that random search can be a reasonable, cheaper substitute for grid search once the number of hyperparameter combinations grows.

Learning curve: Not directly applicable to this study in the standard sense (test accuracy vs. training-set size); Figure 9 is used instead as a proxy to show model stability across different training folds.

Feature importance: Not applicable – neither Naive Bayes nor KNN exposes a native feature-importance score in the way tree-based models do. Figures 2 and 3 give an indirect sense of which features carry signal.

Additional Sensitivity Experiments

Three further checks were run on the tuned KNN configuration to see how sensitive it is to choices outside the main grid.

Train-Test Split	Accuracy	Distance Metric	Accuracy
70–30	0.897	Euclidean	0.898
80–20	0.898	Manhattan	0.908
90–10	0.879		

Weight Type	Accuracy
Uniform	0.898
Distance	0.907

Accuracy is fairly stable across split ratios, dropping only when the training set shrinks to 90:10. Switching the distance metric from Euclidean to Manhattan improves accuracy

by roughly 1 percentage point, and distance-based weighting outperforms uniform weighting by a similar margin – both smaller, single-factor confirmations of the same trends already captured by the full grid search in Section 9.

11 Discussion

Across every experiment in this notebook, the tuned KNN model comes out slightly ahead of the best Naive Bayes variant on accuracy (0.898 vs. 0.876) and on cross-validated accuracy (0.917 vs. 0.886), but the margin is modest and comes with a real cost: KNN takes roughly 10–35 times longer to generate predictions than any Naive Bayes variant, since it has no compact fitted model and must search the stored training data at query time. For a system that needs to classify email in real time at scale, this trade-off matters more than the raw accuracy gap suggests.

Among the three Naive Bayes variants, BernoulliNB clearly outperformed GaussianNB and MultinomialNB. Treating each word/character frequency as a binary presence indicator appears to capture more useful signal for this dataset than modelling the exact continuous value (GaussianNB) or the raw count magnitude (MultinomialNB) – a result that lines up with the historical success of simple presence-based bag-of-words models in spam filtering. GaussianNB’s very high recall (0.959) paired with comparatively low precision (0.715) is a trade-off worth flagging separately: it would flag many legitimate emails as spam, which is usually the more costly mistake in a real filter.

The KDTree and BallTree variants of KNN produced identical accuracy (0.898), as expected, since both structures return the same nearest neighbours and differ only in how they search for them; KDTree fit marginally faster and BallTree predicted marginally faster on this dataset, though neither difference is large enough to matter at this scale. GridSearchCV and RandomizedSearchCV agreed on the same best configuration, with RandomizedSearchCV reaching it in about half the time by sampling only 8 of 16 combinations – a practical argument for trying randomised search first on larger grids. The additional sensitivity checks showed that Manhattan distance and distance-weighted voting both give a small but consistent edge over the defaults (Euclidean distance, uniform voting) at the chosen k .

The main limitations of this study are that most figures rely on a single 80:20 train-test split (partially offset by the 5-fold cross-validation step), that no smoothing or variance parameters were tuned for the Naive Bayes models, and that the KNN timing figures were only measured at this one dataset size. A natural extension would be to test approximate nearest-neighbour libraries or an ensemble of the Naive Bayes and KNN predictions to see whether the accuracy gain from KNN can be retained without its prediction-time cost.

12 Conclusion

This experiment compared three Naive Bayes variants and a tuned KNN classifier on the Spambase dataset. BernoulliNB was the best Naive Bayes model (accuracy 0.876, ROC-AUC 0.950), while KNN tuned via GridSearchCV/RandomizedSearchCV to $k = 3$ with distance weighting and a KD-Tree achieved the best overall accuracy (0.898) and cross-validation accuracy (0.917), at the cost of substantially slower prediction. For a deployment where inference speed matters, BernoulliNB is a strong, much cheaper alternative

that trails the best KNN model by only about 2 accuracy points. The experiment also confirmed that distance-weighted voting and the Manhattan metric both give small, consistent improvements over KNN's defaults, and that randomised hyperparameter search can match exhaustive grid search at a fraction of the compute time.

13 References

https://github.com/sugith-2025/MachineLearning/blob/main/Experiment_2.ipynb